

AI 시대의 프로그래밍: 평가 엔지니어링

김지훈, KAIST AI컴퓨팅학과

오늘날 코딩 에이전트의 한계는 모델의 능력이 아니라 결과물을 평가해 주는 시스템 설계의 부족에 있다. AI는 수십 년간 풀리지 않았던 수학 문제까지 해결하기 시작했지만, 정작 일상적인 프로그래밍에서는 여전히 엉뚱한 결과물을 내놓는다. 수학 증명에서는 AI의 행동을 자동으로 평가하는 환경을 완전히 구축할 수 있지만, 소프트웨어에서는 그렇지 않다. 엄밀한 명세(formal specification)는 평가 기준을 기계가 이해할 수 있는 형태로 만들어 주고, 무엇이 잘못됐는지 더 명확하게 판별할 수 있게 해 준다. 따라서 앞으로 프로그래머가 하는 일은 코드를 직접 쓰는 일에서 평가 아키텍처를 설계하는 일로 옮겨 갈 것이다.

AI의 능력은 푸는 문제에 따라 크게 달라진다. 한쪽에서는 수십 년 동안 풀리지 않았던 수학 문제를 푼다 [2]. 그런데 같은 시기에, 소프트웨어의 작은 기능 하나를 구현하라고 맡기면 여전히 사용자가 원하지 않은 것을 만들어 온다. 몇 번을 고쳐 달라고 해도 조금 어긋난 구현이 돌아올 때도 있다. 이 차이는 난이도로 설명되지 않는다. 수십 년 동안 미해결이었던 문제가 간단한 코드 하나보다 쉬울 리는 없기 때문이다. 같은 모델이 한쪽에서는 잘하고 한쪽에서는 못한다면, 차이는 모델이 아니라 모델이 놓인 환경에 있다고 보는 편이 자연스럽다. 그중에서도 가장 결정적인 차이는 AI가 잘했는지 못했는지를 평가해 주는 시스템의 질에 있다.

AI에게서 원하는 결과물을 얻으려면 두 가지가 필요하다. 하나는 AI에게 일을 전달하기 위해 사용자가 원하는 것을 잘 적는 것이고, 다른 하나는 AI가 내놓은 결과물이 사용자 요구를 만족하는지 평가하고 AI에게 피드백을 주는 것이다. AI가 무엇을 고쳐야 하는지 알려면 무엇이 틀렸는지 알아야 하고, 그 기준은 대개 사람만 알고 있다. 명확한 기준과 평가가 없으면 AI는 그럴듯한 것을 만들어 내는 수밖에 없다.

수학 문제 증명이 AI에게 유리한 이유도 평가 환경에 있다. 수학에서는 명제를 엄밀한 언어(formal language)로 적으면 그것이 그대로 명제가 된다. 즉, 어떤 것을 명세해야 할지가 매우 명확하다. 이에 더해 Lean과 같은 증명 보조기(proof assistant)를 사용하면 기계가 증명이 통과하는지 판정하고 진행이 막힌 위치를 알려 주므로, 평가 장치를 한 번 만들어 두면 모든 문제에 그대로 쓸 수 있다.

하지만 프로그래밍에서는 요구 사항에 무엇을 명세해야 충분한지가 상황마다 달라진다. 수학 증명은 증명하고 싶은 명제를 엄밀한 언어로 적으면 끝이지만 소프트웨어는 신경 써야 할 것이 매번 다르다. 그래서 평가 기준을 프로젝트마다 새로 만들어야 하고, 그렇게 만들어도 미처 생각하지 못한 요구 사항이 항상 나중에 드러난다. 동작은 올바른데 속도가 너무 느릴 수도 있고, 단순히 특정 요구 사항을 빠뜨렸을 수도 있다.

여기서 AI는 적히지 않은 요구 사항을 임의로 정할 수 있다. AI 에이전트는 사용자가 말하지 않은 것을 계속 결정해야 하는데, 그때 멈추고 묻는 대신 그럴듯한 쪽을 골라 진행한다. 많은 경우 그렇게 나온 결과물은 사용자가 처음 제시한 의도에 완전히 부합하지만, 사용자가 명세하지 않은 부분에서 잘못 선택했다더라 그 선택은 평가되지 않는다. 환각 루프(hallucination loop)도 같은 문제에서 비롯된다 [1]. 초반의 실수를 인지하지 못하고 계속 나아가면, 단계마다 진전이 있는 것처럼 보이지만 정작 원하던 것에서는 계속 멀어진다. 이는 더 일찍 이루어졌어야 할 평가가 미루어졌기 때문이다.

따라서 코딩 에이전트가 더 발전하기 위해서는 모델을 더 똑똑하게 만드는 것보다 평가 시스템을 개선하는 것이 더 중요하다. 이미 AI는 충분히 똑똑하고, 부족한 건 AI의 행동을 평가할 아키텍처의 설계다.

코딩 에이전트를 평가하기 위해서는 먼저 평가할 기준, 즉 명세가 있어야 한다. 앞서 언급했듯이 프로그래밍에서 명세하고 싶은 것은 상황마다 다르다. 메모리 안전성(memory safety)을 명세하고 싶을 수도 있고, 프로그램 전체 동작을 완전히 명세하고 싶을 수도 있다. 이는 프로그램의 역할이나 규모, 중요성에 따라 달라진다.

중요한 것은 명세라는 기준이 있다면 명세와 구현 중 어느 것이 잘못됐는지가 더 명확해진다는 점이다. 무언가 잘못되었을 때 코드가 명세를 안 지킨 건지, 명세 자체를 내가 잘못 적은 건지 훨씬

분명하게 가릴 수 있다. 그러면 코딩 에이전트도 “고치겠습니다”와 “주어진 조건 자체가 이상한 것 같은데 맞나요?”를 구분할 수 있다. AI가 후자 대신 전자를 말하는 경우가 많은 것은 후자를 말할 근거가 없기 때문이다. 이에 더해 사용자 또한 AI가 의도를 잘못 이해한 것인지 구현을 잘못된 것인지 구분할 수 있어 평가도 더 명확해진다. 명세가 처음부터 완벽할 필요도 없고, AI와 대화하며 조금씩 채워 나가면 된다.

따라서 앞으로 프로그래머의 일은 코딩 에이전트를 어떻게 평가할지 설계하는 것으로 바뀔 것이라 생각한다. 이는 평가의 기준이 될 명세의 범위를 정하고, 실제로 명세를 작성하고, AI가 명세를 어떻게 이용할지 설계하고, AI와 함께 평가 기준을 만들어 나가는 것이다. 이것이 완전히 새로운 일은 아니다. 원래 프로그래머가 하던 개발 주기 관리나 아키텍처 설계도 문제의 범위를 좁히고 원인을 확실하게 하기 위한 시스템을 설계하는 것이었다. 달라지는 건 문제를 사람이 아닌 AI가 만들어 낸다는 점이다. 사람을 위해 아키텍처를 설계하고 개선하던 일이 이제는 AI를 위한 평가 기준과 시스템을 만드는 일로 옮겨 가는 것뿐이다.

References

- [1] Bertrand Meyer. 2026. Artificial Intelligence for Software Engineering: From Probable to Provable. *Commun. ACM* 69, 6 (2026), 46–49.
- [2] OpenAI. 2026. Ten Advances in Mathematics and Theoretical Computer Science. <https://openai.com/index/ten-advances-in-mathematics/>